EDA & Stats

Explore any dataset in the right order. Pick the right test. Say what it means.

DATAMOUS

Cheat Sheet

Every line tested on pandas, seaborn, SciPy and statsmodels

The first hour with any dataset. Four steps, in order: each one catches the problems that would spoil the next.

1 Understand what you have

ASK	CODE	LOOK FOR
Load dates as dates	`df = pd.read_csv('orders.csv', parse_dates=['order_date'])`	Date columns show `datetime64`, not `object`
Size and kind of each column	`df.info()` `df.nunique()`	What one row is; whether each column is a number, category, date or ID
Look at real rows	`df.sample(5, random_state=1)`	Rows from the middle, not just the top: odd codes, units
Blanks per column	`df.isna().sum()`	How complete each column is
Did the join keep the rows?	`m = orders.merge(customers, on='customer_id', how='left', validate='many_to_one')`	`len(m) == len(orders)`; an error means repeated keys

2 One column at a time

ASK	CODE	LOOK FOR
Summarize every number	`df.describe()`	Min and max that can't be real; mean far from the median
Draw the shape	`sns.histplot(data=df, x='revenue')`	One hump or two, long tail, a wall at zero
How skewed?	`df['revenue'].skew()`	Beyond ±1 is strongly skewed: the median and IQR describe it better
Count every category	`df['plan'].value_counts(dropna=False)`	Blanks, typos, very small groups
Find unusual values	`q1, q3 = df['revenue'].quantile([.25, .75])` `high = df[df['revenue'] > q3 + 1.5 * (q3 - q1)]`	Some are errors, some are real but rare values
Do blanks follow a pattern?	`df['rating'].isna().groupby(df['plan']).mean()`	One group missing far more often than the others
Repeated rows	`df.duplicated().sum()`	Double-counted orders inflate every total

3 Two columns

ASK	CODE	LOOK FOR
Scatter before correlation	`sns.scatterplot(data=df, x='items', y='revenue')`	Straight line, curve, clusters, a few far-out points
How strong is the link?	`df['items'].corr(df['revenue'], method='spearman')`	Pearson for straight lines, Spearman for curves and outliers; under 0.3 is weak
Compare groups, with counts	`df.groupby('region')['revenue'].describe()`	Count, mean and median side by side; tiny groups with extreme averages
Compare whole shapes	`sns.boxplot(data=df, x='region', y='revenue')`	Gaps in the middle and in the spread
Two categories	`pd.crosstab(df['region'], df['plan'], normalize='index')`	`'index'`: shares within each row add up to 100%

4 Many columns and time

ASK	CODE	LOOK FOR
Main numbers together	`sns.pairplot(df[['revenue', 'items', 'tenure']])`	Every pair of numbers in one grid
Does a gap hold in every group?	`df.pivot_table(index='plan', columns='region', values='revenue', aggfunc='mean')`	A gap can shrink or flip inside groups (Simpson's paradox)
Numbers over time	`monthly = df.resample('MS', on='order_date')\` `['revenue'].sum()`	Trend, season, one-off spikes
Trend or noise?	`monthly.rolling(3).mean()`	Smooths out noise; the same month across years shows seasons

`df` = your table · `a`, `b`, `c` = one column's values for each group · `pd`, `np`, `sns`, `stats` (`from scipy import stats`) = the usual imports

5 Which test?

Each test answers one question for one kind of data. The last column is what goes with the p-value, usually the size of the effect.

One group, or the same people twice

QUESTION	DATA	CODE	GOES WITH IT
Is the data close to normal?	One numeric column	`stats.shapiro(a)`	Big samples fail it even for tiny departures from normal
Does the average differ from a target?	One numeric column	`stats.ttest_1samp(a, 30)`	The gap from the target, with its interval
Did the same people score differently?	Paired, bell-shaped gaps	`stats.ttest_rel(before, after)`	The mean change, with its interval
Same, but skewed or ratings	Paired	`stats.wilcoxon(before, after)`	How many got better, worse, same

Two or more separate groups

QUESTION	DATA	CODE	GOES WITH IT
Do two averages differ?	Two groups	`stats.ttest_ind(a, b, equal_var=False)`	The gap with its interval; Cohen's d
Same, but skewed or outliers	Two groups	`stats.mannwhitneyu(a, b)`	The medians; `statistic / (n1 * n2)` = "a is bigger X% of the time"
Do several averages differ?	3+ groups	`stats.f_oneway(a, b, c)`	The gaps between groups
Same, but skewed or ratings	3+ groups	`stats.kruskal(a, b, c)`	Each group's median
Which pairs differ?	After `f_oneway()`	`pairwise_tukeyhsd(df['revenue'], df['region'])`	Gaps with intervals, already adjusted
Do two rates differ?	Yes/no, two groups	`proportions_ztest(successes, sizes)`	The gap in percentage points, with its interval

Links between columns

QUESTION	DATA	CODE	GOES WITH IT
Are two categories linked?	A table of counts	`stats.chi2_contingency(pd.crosstab(df['region'], df['plan']))`	The gap in shares; Cramér's V
Do two numbers move together?	Two numeric columns	`stats.pearsonr(df['items'], df['revenue'])`	r itself, and the scatter plot
Same, for ranks or curves	Ratings, curves	`stats.spearmanr(df['items'], df['revenue'])`	Spearman's rho, and the scatter plot
How much does each factor matter?	Numeric outcome	`smf.ols('revenue ~ items + C(region)', data=df).fit()`	Each coefficient, with `.conf_int()`
Same, for a yes/no outcome	A 0/1 column	`smf.logit('churned ~ tenure + C(plan)', data=df).fit()`	Each coefficient, with `.conf_int()`
Ran many tests?	A list of p-values	`multipletests(p_values, method='holm')[1]`	Adjusted p-values

`smf` = `import statsmodels.formula.api as smf` · `proportions_ztest` from `statsmodels.stats.proportion` · `pairwise_tukeyhsd` from `statsmodels.stats.multicomp` · `multipletests` from `statsmodels.stats.multitest`

6 A/B tests

Random groups, one change, one planned look. The steps in order:

STEP	CODE	WHAT IT TELLS YOU

1 Plan the size first	`effect = proportion_effectsize(0.09, 0.08)`
	`NormalIndPower().solve_power(effect,`
	`  alpha=0.05, power=0.8)`
		Visitors needed per group to spot 8% → 9% four times out of five.

2 Check the split	`sizes = ab['group'].value_counts()`
	`2 * stats.binom.cdf(sizes.min(),`
	`  n=sizes.sum(), p=0.5)`
		For a 50/50 test. A small p-value (a sample ratio mismatch) means something in the setup broke.

3 Compare the rates	`buyers = ab.groupby('group')['bought'].agg(`
	`  ['sum', 'count', 'mean'])`
	`proportions_ztest(buyers['sum'], buyers['count'])`
		Needs at least 10 buyers and 10 non-buyers in each group.

4 Size the gain	`new, old = buyers.loc['new'], buyers.loc['control']`
	`confint_proportions_2indep(new['sum'], new['count'],`
	`  old['sum'], old['count'])`
		The interval for new minus control, in percentage points. Relative lift = gap / control rate.

5 Compare amounts	`stats.ttest_ind(ab.loc[ab['group'] == 'new', 'revenue'],`
	`  ab.loc[ab['group'] == 'control', 'revenue'],`
	`  equal_var=False)`
		Revenue per visitor: Welch's t-test, with the gap and its interval.

6 Many metrics or segments	`multipletests(p_values, method='holm')[1]`	Each extra test adds a chance of a false win; adjusted p-values account for that.

`ab` = one row per visitor: `group`, `bought` (0/1), `revenue` · imports from `statsmodels.stats.proportion` and `statsmodels.stats.power`

Peeking
In a 4-week test, checking the p-value daily and stopping at the first p < 0.05 raises false alarms from 1 in 20 to about 1 in 4.

Whole weeks
Weekday and weekend visitors can behave differently; whole weeks include both.

Random groups
Random assignment is what lets a test show cause. Without it, a gap shows a link.

Read the result

p-value	How surprising the gap would be if nothing were going on. It's compared with a significance level set in advance, usually 0.05.
p < 0.05	Evidence of a difference. It says nothing about how big or important the gap is.
p ≥ 0.05	Not enough evidence of a difference, which is not the same as no difference.
Interval	The range of sizes the true gap could plausibly have. One that crosses 0 includes no gap at all.
Effect size	How big the gap is. Cohen's d: about 0.2 small, 0.5 medium, 0.8 large.

Cohen's d for two groups

```
def cohens_d(a, b):
    pooled_var = (((len(a) - 1) * a.var() + (len(b) - 1) * b.var())
                  / (len(a) + len(b) - 2))
    return (a.mean() - b.mean()) / np.sqrt(pooled_var)
```

The gap between two means, in standard deviations. Pairs with `ttest_ind()`.